IEP DESK

Product Development Plan

Complete Technical & Engineering Specification

Version	1.0.0
Author	Nuraj Khan Ramin
Product	IEP Desk
Stack	MERN + OpenRouter + Firebase
Budget	**$0 — Fully Free**
Hosting	Vercel + Render.com

CONFIDENTIAL — For Development Use Only

1. Product Overview

IEP Desk is a free AI-powered web application that helps parents of children with learning disabilities navigate the American special education system. Parents upload their child's IEP document and receive a plain English analysis, legal rights breakdown, and a personalized meeting preparation kit — all powered by AI.

1.1 Problem Statement

7.5 million children in the US have Individualized Education Programs (IEPs). These are legal documents guaranteeing specialized educational support. However:

- Documents are 20+ pages of dense legal jargon that most parents cannot understand
- Parents attend IEP meetings outnumbered by school specialists and feel powerless
- 40% of IEP parents report feeling excluded from the process legally designed to include them
- No dedicated SaaS product exists specifically for parents — only tools for schools and teachers

1.2 Solution

IEP Desk gives parents three core capabilities:

1. Upload IEP document → AI analyzes in plain English, flags red flags and missing accommodations
2. Enter child diagnosis and state → AI shows exact legal rights and accommodations entitled to
3. Before any meeting → AI generates personalized questions, pushback scripts, and email templates

1.3 Target Users

User Type	Description	Why They Use IEP Desk
Primary Parents	Parents of IEP/504 children	Understand rights, prepare for meetings
Secondary Advocates	IEP advocates, consultants	Tool for client preparation
Tertiary Schools	School districts (future)	White-label licensing

2. Technology Stack

Every tool in this stack is 100% free with no credit card required. The entire system costs $0 per month at MVP scale.

2.1 Complete Stack Overview

Layer	Tool	Purpose	Free Tier
Frontend	React.js (Vite)	User interface — all parent-facing pages	Free forever
Backend	Node.js + Express	API server — all business logic	Free forever
Database	MongoDB Atlas	Store users, profiles, analyses	512MB free
Auth	Firebase Auth	User login — Google, email/password	Free forever
AI	OpenRouter API	IEP analysis, rights engine, meeting prep	Free models
File Storage	Cloudinary	PDF and image uploads from users	25GB free
Frontend Host	Vercel	Deploy and serve React app	Free tier
Backend Host	Render.com	Deploy and serve Express API	Free tier
Email	Resend.com	Transactional emails to users	3,000/month free
PDF Parse	PDF.js (Mozilla)	Extract text from uploaded IEP PDFs	Open source
Styling	Tailwind CSS	All UI styling — minimal and clean	Free forever

2.2 AI Model — OpenRouter

OpenRouter provides access to multiple free AI models with no credit card required. Use the following models in priority order:

Model	Use Case	Limits
meta-llama/llama-4-maverick:free	Primary — IEP analysis, all complex tasks	200 req/day, 20/min

google/gemini-2.0-flash-exp:free	Backup — rights engine, fast responses	200 req/day, 20/min
mistral/mistral-7b-instruct:free	Emergency fallback if others unavailable	200 req/day, 20/min

> ⚠️ **IMPORTANT — Rate Limiting**
> Free OpenRouter models allow 200 requests per day and 20 per minute.
> Always implement retry logic with exponential backoff in the API service.
> Cache AI responses in MongoDB so the same IEP is never analyzed twice.
> If primary model fails, automatically fall back to the next model in the list.

2.3 Image & File Storage — Cloudinary

Cloudinary provides free file storage with no credit card required. Sign up with Google account. The free plan gives 25GB storage and 25GB bandwidth per month — more than enough for MVP.

- Store all uploaded IEP PDFs as raw files in Cloudinary
- Store user profile photos as images in Cloudinary
- Organize files in folders by user ID: iep-desk/{userId}/
- Always save the Cloudinary public_id in MongoDB for future deletion

3. System Architecture

3.1 High Level Architecture

System Flow

User Browser (React) → Vercel CDN → React App
React App → Firebase Auth → Get ID Token
React App → Render.com → Express API (with Firebase token in header)
Express API → MongoDB Atlas → Read/Write data
Express API → OpenRouter API → AI analysis
Express API → Cloudinary → Upload/retrieve files
Express API → Resend.com → Send emails
Express API → React App → Return JSON response

3.2 Authentication Flow

Authentication uses Firebase Auth on the frontend and Firebase Admin SDK on the backend to verify tokens.

4. User clicks Login/Signup on React frontend
5. Firebase Auth handles authentication — Google OAuth or email/password
6. Firebase returns an ID token to the React app
7. React stores token and sends it in every API request as Authorization: Bearer {token}
8. Express middleware verifies token using Firebase Admin SDK on every protected route
9. If valid, middleware attaches user object to req.user and continues to route handler
10. If invalid, middleware returns 401 Unauthorized immediately

3.3 File Upload Flow

11. User selects PDF file in React frontend
12. React uses PDF.js to extract text from the PDF client-side
13. React sends both the file (FormData) and extracted text (JSON) to Express API
14. Express receives the file in memory using Multer (memory storage — no disk writes)
15. Express uploads file buffer to Cloudinary using upload_stream
16. Cloudinary returns a secure_url and public_id
17. Express saves secure_url and public_id to MongoDB Document model
18. Express sends extracted text to OpenRouter for AI analysis
19. AI result is saved to MongoDB Analysis model
20. Express returns full analysis result to React frontend

3.4 AI Analysis Flow

21. Express receives IEP text + child profile data
22. Express constructs the analysis prompt using the profile data as context
23. Express calls OpenRouter API with primary model (llama-4-maverick)
24. OpenRouter returns AI response as JSON string
25. Express parses JSON — if parsing fails, retries with regex extraction
26. If primary model fails, Express retries with backup model (gemini-2.0-flash)
27. Parsed result is validated — all required fields must be present
28. Valid result is saved to MongoDB and returned to frontend

4. Project Structure

The project uses a monorepo structure with two separate folders — client for React frontend and server for Node.js backend. Both are deployed independently.

4.1 Root Structure

Root Level

```
iep-desk/
├── client/        → React frontend (deployed to Vercel)
├── server/        → Node.js + Express backend (deployed to Render.com)
├── .gitignore
└── README.md
```

4.2 Client Structure (React)

client/src/ folder structure

```
src/
├── components/
│   ├── ui/              → Reusable: Button, Input, Card, Modal, Badge, Spinner
│   ├── layout/          → Navbar, Sidebar, Footer, PageWrapper
│   ├── dashboard/       → ProfileCard, StatsCard, QuickActionCard
│   ├── analyzer/        → DocumentUpload, AnalysisResult, GoalCard, RedFlagCard
│   ├── rights/          → RightsCard, AccommodationCard, QuestionCard
│   └── meeting/         → QuestionList, PushbackScript, EmailTemplate, Checklist
├── pages/
│   ├── Landing.jsx      → Public landing page
│   ├── Login.jsx        → Login page
│   ├── Register.jsx     → Register page
│   ├── Dashboard.jsx    → Main user dashboard after login
│   ├── Profile.jsx      → Create/edit child profile
│   ├── Analyzer.jsx     → Upload IEP and view analysis
│   ├── Rights.jsx       → View legal rights for child's profile
│   ├── MeetingPrep.jsx  → Meeting prep kit generator
│   ├── Documents.jsx    → Document vault — view all uploads
│   └── Settings.jsx     → User account settings
├── admin/               → Admin panel pages (role-protected)
│   ├── AdminDashboard.jsx  → Stats overview
│   ├── AdminUsers.jsx      → View, search, manage all users
│   ├── AdminAnalyses.jsx   → View all IEP analyses
│   └── AdminSettings.jsx   → App-wide settings
├── context/
```

```
│   ├── AuthContext.jsx      → Firebase auth state — user, loading, login, logout
│   └── ProfileContext.jsx   → Active child profile state
├── hooks/
│   ├── useAuth.js           → Access auth context
│   ├── useProfile.js        → Access profile context
│   └── useApi.js            → Authenticated API call helper
├── utils/
│   ├── pdfParser.js         → PDF.js text extraction utility
│   ├── api.js               → Base fetch wrapper with auth header
│   └── constants.js         → DIAGNOSES, STATES, GRADES arrays
├── App.jsx                  → Router setup with protected routes
└── main.jsx                 → React entry point
```

4.3 Server Structure (Node.js)

server/ folder structure

```
server/
├── config/
│   ├── db.js                → MongoDB Atlas connection
│   ├── firebase.js          → Firebase Admin SDK setup
│   └── cloudinary.js        → Cloudinary SDK configuration
├── models/
│   ├── User.js              → User schema
│   ├── Profile.js           → Child profile schema
│   ├── Analysis.js          → IEP analysis result schema
│   ├── Document.js          → Document vault schema
│   └── MeetingPrep.js       → Meeting prep result schema
├── routes/
│   ├── auth.routes.js       → POST /api/auth/me
│   ├── profile.routes.js    → CRUD /api/profile
│   ├── analyze.routes.js    → POST /api/analyze
│   ├── rights.routes.js     → POST /api/rights
│   ├── meeting.routes.js    → POST /api/meeting-prep
│   ├── document.routes.js   → CRUD /api/documents
│   └── admin.routes.js      → GET /api/admin/*
├── controllers/
│   ├── auth.controller.js
│   ├── profile.controller.js
│   ├── analyze.controller.js
│   ├── rights.controller.js
│   ├── meeting.controller.js
│   ├── document.controller.js
│   └── admin.controller.js
├── middleware/
```

```
│   ├── auth.middleware.js    → Verify Firebase token, attach req.user
│   ├── admin.middleware.js   → Check req.user.role === 'admin'
│   └── upload.middleware.js  → Multer memory storage for file uploads
├── prompts/
│   ├── analyzeIEP.js         → Full IEP analysis prompt builder
│   ├── rightsEngine.js       → Legal rights prompt builder
│   └── meetingPrep.js        → Meeting prep prompt builder
├── services/
│   ├── openrouter.service.js → AI API calls with fallback and retry
│   ├── cloudinary.service.js → File upload and delete helpers
│   └── email.service.js      → Resend.com email sending
├── app.js                    → Express app setup, middleware, routes
└── package.json
```

5. Database Schemas (MongoDB)

All data is stored in MongoDB Atlas. Use Mongoose for schema definition and validation. Below are the complete field specifications for each collection.

5.1 User Schema

Field	Type	Required	Notes
firebaseUid	String	Yes — unique	From Firebase Auth token
email	String	Yes — unique	User email address
name	String	Yes	Display name
avatar	String	No	Cloudinary URL — null by default
role	Enum	No	'user' or 'admin' — default 'user'
plan	Enum	No	'free', 'beta', 'paid' — default 'free'
createdAt	Date	Auto	Auto-set on creation
lastActive	Date	Auto	Updated on every authenticated request

5.2 Profile Schema (Child Profile)

Field	Type	Required	Notes
userId	ObjectId	Yes	Reference to User._id
childName	String	Yes	Child's first name
dateOfBirth	Date	No	Optional — for age calculation
grade	String	Yes	e.g. '3rd Grade', 'Kindergarten'
school	String	No	School name
schoolDistrict	String	No	School district name
state	String	Yes	US state — used for state-specific rights
diagnoses	[String]	Yes	Array — e.g. ['ADHD', 'Dyslexia']
planType	Enum	No	'IEP', '504', 'Both', 'None' — default 'IEP'

notes	String	No	Parent's private notes

5.3 Analysis Schema (IEP Analysis Results)

Field	Type	Required	Notes
userId	ObjectId	Yes	Reference to User._id
profileId	ObjectId	Yes	Reference to Profile._id
documentUrl	String	No	Cloudinary secure_url of the PDF
documentName	String	No	Original filename of upload
rawText	String	No	Full extracted text — for re-analysis
result	Object	Yes	Full JSON result from AI — see AI Response spec
createdAt	Date	Auto	Timestamp of analysis

5.4 Document Schema (Document Vault)

Field	Type	Required	Notes
userId	ObjectId	Yes	Reference to User._id
profileId	ObjectId	Yes	Reference to Profile._id
name	String	Yes	Document display name
type	Enum	Yes	'IEP', '504', 'Evaluation', 'Email', 'Meeting Notes', 'Other'
url	String	Yes	Cloudinary secure_url
publicId	String	Yes	Cloudinary public_id — needed for deletion
fileSize	Number	No	File size in bytes
notes	String	No	Parent notes about this document
date	Date	No	Date of the document — not upload date

6. API Endpoints

All endpoints except /api/auth/me require a valid Firebase ID token in the Authorization header as: Authorization: Bearer {token}. Admin endpoints additionally require the user's role to be 'admin'.

6.1 Auth Endpoints

Method	Route	Auth	Description
POST	/api/auth/me	Required	Get or create user in MongoDB from Firebase token

6.2 Profile Endpoints

Method	Route	Auth	Description
GET	/api/profile	Required	Get all child profiles for authenticated user
POST	/api/profile	Required	Create a new child profile
GET	/api/profile/:id	Required	Get a single profile by ID (must belong to user)
PUT	/api/profile/:id	Required	Update a profile (must belong to user)
DELETE	/api/profile/:id	Required	Delete profile and all associated data

6.3 AI Feature Endpoints

Method	Route	Auth	Description
POST	/api/analyze	Required	Upload PDF + analyze with AI. Body: multipart/form-data with file and profileId
GET	/api/analyze/:profileId	Required	Get all past analyses for a specific profile
POST	/api/rights	Required	Get legal rights for child profile. Body: { profileId }

POST	/api/meeting-prep	Required	Generate meeting prep kit. Body: { profileId, analysisId, meetingType }

6.4 Document Vault Endpoints

Method	Route	Auth	Description
GET	/api/documents/:profileId	Required	Get all documents for a profile
POST	/api/documents	Required	Upload a new document. Body: multipart/form-data
DELETE	/api/documents/:id	Required	Delete document from MongoDB AND Cloudinary

6.5 Admin Endpoints

Method	Route	Auth	Description
GET	/api/admin/stats	Admin only	Total users, analyses, today signups, active users
GET	/api/admin/users	Admin only	Paginated list of all users with search
PATCH	/api/admin/users/:id	Admin only	Update user role or plan
DELETE	/api/admin/users/:id	Admin only	Delete user and all their data
GET	/api/admin/analyses	Admin only	View all analyses across all users

7. AI Prompt Specifications

The prompts are the most critical part of the product. They must be followed exactly. All prompts must instruct the AI to return valid JSON only — no markdown, no code blocks, no extra text. Always include the child's diagnosis and state as context.

7.1 IEP Analysis Prompt — Required JSON Response Structure

analyzeIEP prompt must produce this exact JSON structure:

```
{
  "summary": "string — 2-3 sentence plain English summary",
  "overallScore": {
    "score": number (1-10),
    "explanation": "string — one sentence why this score"
  },
  "currentLevel": {
    "summary": "string — where is child right now",
    "concerns": ["string", "string"]
  },
  "goalsAnalysis": [{
    "goal": "exact goal text as written",
    "quality": "strong OR weak OR needs improvement",
    "problem": "string — what is wrong, null if none",
    "improvedVersion": "string — better version",
    "plainEnglish": "string — what this means simply"
  }],
  "servicesAnalysis": [{
    "service": "string", "frequency": "string",
    "concern": "string or null", "plainEnglish": "string"
  }],
  "accommodationsAnalysis": {
    "provided": ["string"],
    "missing": ["string — accommodations not present but needed"],
    "explanation": "string"
  },
  "redFlags": [{
    "issue": "string", "whyItMatters": "string", "whatToDo": "string"
  }],
  "actionItems": ["string"],
  "disclaimer": "This is informational only, not legal advice."
}
```

7.2 Rights Engine Prompt — Required JSON Response Structure

rightsEngine prompt must produce this exact JSON structure:

```
{
  "federalRights": [{
    "right": "string", "description": "string", "howToRequest": "string"
  }],
  "commonAccommodations": [{
    "accommodation": "string",
    "description": "string",
    "whoBenefits": "string — why this helps this specific diagnosis"
  }],
  "stateSpecific": [{
    "right": "string", "description": "string"
  }],
  "redFlags": ["string — common violations to watch for"],
  "questionsToAsk": ["string — specific meeting questions"],
  "disclaimer": "This is informational only, not legal advice."
}
```

7.3 Meeting Prep Prompt — Required JSON Response Structure

meetingPrep prompt must produce this exact JSON structure:

```
{
  "meetingOverview": "string — 2-3 sentences what to expect",
  "keyQuestions": [{
    "question": "string — exact words to say",
    "whyImportant": "string",
    "goodAnswer": "string — what school should say",
    "redFlagAnswer": "string — what to watch out for"
  }],
  "pushbackScripts": [{
    "situation": "string — when school says X",
    "whatToSay": "string — exact words",
    "legalBasis": "string — the law that supports this"
  }],
  "rightsReminder": ["string"],
  "emailTemplates": {
    "beforeMeeting": { "subject": "string", "body": "string" },
    "afterMeeting": { "subject": "string", "body": "string" },
    "ifSchoolSaysNo": { "subject": "string", "body": "string" }
  },
  "checklist": {
    "bringToMeeting": ["string"],
    "beforeMeeting": ["string"],
```

```
    "afterMeeting": ["string"]
  },
  "disclaimer": "This is informational only, not legal advice."
}
```

7.4 AI Service Error Handling Rules

The OpenRouter service must follow these rules exactly:

29. Always try the primary model first: meta-llama/llama-4-maverick:free
30. If primary model returns an error, automatically retry with: google/gemini-2.0-flash-exp:free
31. If backup model also fails, retry with: mistral/mistral-7b-instruct:free
32. Parse the AI response as JSON — if JSON.parse fails, use regex to extract the JSON object
33. If all models fail after 3 total attempts, return a 503 error to the client
34. Cache successful AI responses in MongoDB — never call AI twice for the same document
35. Always include the disclaimer field in every response before returning to frontend

8. Pages & UI Specification

All pages use a minimal, clean design. Primary color: #2563EB (blue). Background: #F9FAFB (light gray). Cards: white with subtle shadow. Font: system-ui or Inter. No heavy animations. Mobile-first responsive layout.

8.1 Public Pages (No Auth Required)

Landing Page — /

- Hero section: headline, subheadline, single CTA button 'Join Waitlist — Free'
- Stats bar: '7 million+ IEP children', '40% parents feel excluded', '5 min to understand'
- Pain section: 3 cards — Confusion, Fear, Chaos with emoji and description
- How it works: 3 numbered steps with icons
- Features grid: 6 feature cards with icon, title, description
- Testimonials: 2 real quotes from validation interviews
- Pricing: Beta access — free for first 100 families, list features, CTA button
- Footer: links to How It Works, Join Waitlist, Terms, Privacy

Login Page — /login

- Centered card with IEP Desk logo
- Google Sign-In button — primary action
- Email/password form — secondary action
- Link to register page

Register Page — /register

- Same layout as login
- Name, email, password fields
- Google Sign-Up button
- Link to login page

8.2 Protected Pages (Auth Required)

Dashboard — /dashboard

- Welcome header with user's first name
- 3 quick action cards: Analyze IEP, View Rights, Meeting Prep — each links to respective page
- Active child profile card showing name, grade, diagnoses, state

- Recent analyses list — last 3 analyses with date and score
- If no profile exists — show 'Create Profile' prompt card

Profile — /profile

- Form to create or edit child profile
- Fields: child name, date of birth (optional), grade (dropdown), state (dropdown), school, district, diagnoses (multi-select chips), plan type (IEP/504/Both)
- Diagnoses options: ADHD, Autism Spectrum Disorder, Dyslexia, Dysgraphia, Dyscalculia, Speech/Language Delay, Intellectual Disability, Emotional Behavioral Disorder, Traumatic Brain Injury, Visual Impairment, Hearing Impairment, Other Learning Disability
- Save button — POST or PUT to /api/profile
- Allow multiple child profiles — show list of existing profiles at top

Analyzer — /analyzer

- Step 1: Select child profile (dropdown if multiple exist)
- Step 2: PDF upload zone — drag and drop or click to browse
- Step 3: When file selected — show filename and 'Analyze IEP' button
- Loading state: spinner with message 'Analyzing your IEP — this takes about 30 seconds...'
- Results section shows after analysis completes:
  - Overall score card (1-10) with color coding — red <4, yellow 4-6, green >7
  - Summary card — plain English overview
  - Red Flags section — orange/red cards for each red flag with issue, why it matters, what to do
  - Goals Analysis — card for each goal with quality badge, problem, improved version
  - Missing Accommodations — yellow warning section listing gaps
  - Action Items — numbered list of specific next steps
  - Button at bottom: 'Generate Meeting Prep Kit' → links to /meeting-prep with analysisId

Rights — /rights

- Select child profile (dropdown)
- Click 'Show My Rights' button
- Loading state while AI generates
- Results display:
  - Federal Rights section — accordion list, each item shows right name, description, how to request
  - Common Accommodations — grid of cards by diagnosis
  - State-Specific Rights — list for selected state

    - Red Flags to Watch — warning list
    - Questions to Ask — numbered list of specific questions
    - Disclaimer at bottom in gray small text

Meeting Prep — /meeting-prep

- Select child profile and past analysis (dropdowns)
- Select meeting type: Annual Review, Evaluation Meeting, Amendment Meeting, Initial IEP, Other
- Click 'Generate Meeting Prep Kit'
- Results display:
    - Meeting Overview card — what to expect
    - Key Questions list — each with question, why important, good/red flag answers
    - Pushback Scripts — accordion, each with situation, what to say, legal basis
    - Rights Reminder — bulleted list
    - Email Templates — tabs for Before/After/If School Says No with copy button
    - Checklist — 3 sections: bring to meeting, before meeting, after meeting with checkboxes
    - Print button at top right for entire prep kit

Documents — /documents

- Select child profile
- Upload button — opens file picker for PDF, images, or other documents
- Document type selector: IEP, 504, Evaluation, Email, Meeting Notes, Other
- Document name field and optional date field
- Grid/list of all uploaded documents with name, type, date, file size
- Each document has View button (opens Cloudinary URL) and Delete button
- Search bar to filter documents by name or type

Settings — /settings

- Update display name
- Upload profile photo — stored in Cloudinary
- Change email (Firebase)
- Change password (Firebase)
- Delete account button — deletes all user data from MongoDB and Cloudinary

8.3 Admin Panel Pages (Admin Role Required)

Admin Dashboard — /admin

- Stats cards: Total Users, Total Analyses, Today Signups, Active This Week
- Simple bar chart of signups by day over last 30 days — use recharts library
- Recent signups list — last 10 users with name, email, join date

Admin Users — /admin/users

- Search bar to find users by name or email
- Paginated table: Name, Email, Plan, Role, Joined, Last Active, Actions
- Actions: Change Role (user/admin), Change Plan (free/beta/paid), Delete User
- Each row clickable to see user's profiles and analyses

Admin Analyses — /admin/analyses

- Table of all analyses: User Email, Child Name, State, Score, Date
- Click row to see full analysis result
- Filter by date range or score range

9. Environment Variables

Store all secrets in environment variable files. Never commit these to Git. Add all variables to both local .env files and the deployment platform dashboards.

9.1 Server Environment Variables (.env in /server)

Variable	Where to Get It	Example Value
MONGODB_URI	MongoDB Atlas	mongodb+srv://user:pass@cluster.mongodb.net/iepdesk
OPENROUTER_API_KEY	openrouter.ai	sk-or-v1-xxxxxxxxxxxx
CLOUDINARY_CLOUD_NAME	Cloudinary dashboard	your_cloud_name
CLOUDINARY_API_KEY	Cloudinary dashboard	123456789012345
CLOUDINARY_API_SECRET	Cloudinary dashboard	xxxxxxxxxxxxxxxxxxxx
FIREBASE_PROJECT_ID	Firebase console	iep-desk-xxxxx
FIREBASE_PRIVATE_KEY	Firebase service account	-----BEGIN PRIVATE KEY-----...
FIREBASE_CLIENT_EMAIL	Firebase service account	firebase-adminsdk@project.iam...
RESEND_API_KEY	resend.com	re_xxxxxxxxxxxx
CLIENT_URL	Your Vercel URL	https://iepdesk.vercel.app
PORT	Any port number	5000

9.2 Client Environment Variables (.env in /client)

Variable	Where to Get It	Example Value
VITE_API_URL	Your Render.com URL	https://iep-desk-api.onrender.com
VITE_FIREBASE_API_KEY	Firebase console	AIzaSy-xxxxxxxxxxxx
VITE_FIREBASE_AUTH_DOMAIN	Firebase console	iep-desk-xxxxx.firebaseapp.com
VITE_FIREBASE_PROJECT_ID	Firebase console	iep-desk-xxxxx

VITE_FIREBASE_STORAGE_BUCKET	Firebase console	iep-desk-xxxxx.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID	Firebase console	123456789012
VITE_FIREBASE_APP_ID	Firebase console	1:123456789012:web:xxxx

10. Setup & Deployment Guide

Follow these steps in exact order. Do not skip any step. Each service must be set up before starting development.

10.1 Service Setup (Do This First)

Step 1 — MongoDB Atlas

36. Go to mongodb.com/atlas → sign up free with Google
37. Create a new project called 'iep-desk'
38. Create a free cluster (M0 Sandbox — 512MB)
39. Create a database user with username and password
40. Add 0.0.0.0/0 to IP allowlist (allow all)
41. Get connection string — replace <password> with your password
42. Save as MONGODB_URI in server .env file

Step 2 — Firebase

43. Go to console.firebase.google.com → create new project 'iep-desk'
44. Enable Authentication → Sign-in methods → Enable Google and Email/Password
45. Go to Project Settings → General → scroll to 'Your apps' → Add Web App
46. Copy the Firebase config object → these become VITE_FIREBASE_* env vars
47. Go to Project Settings → Service Accounts → Generate new private key
48. Download JSON file → extract project_id, private_key, client_email → save as FIREBASE_* env vars

Step 3 — OpenRouter

49. Go to openrouter.ai → sign up free with Google
50. Go to Keys → Create new API key
51. Save as OPENROUTER_API_KEY in server .env
52. Free models are available immediately — no credit card needed

Step 4 — Cloudinary

53. Go to cloudinary.com → sign up free with Google
54. Dashboard shows Cloud Name, API Key, API Secret
55. Save all three as CLOUDINARY_* env vars in server .env
56. No credit card required — free plan gives 25GB storage

Step 5 — Resend

57. Go to resend.com → sign up free
58. Add and verify your domain OR use onboarding email for testing
59. Go to API Keys → Create API Key
60. Save as RESEND_API_KEY in server .env

10.2 Local Development Setup

Initial Setup

Run these commands in order:

```
# Clone or create project
mkdir iep-desk && cd iep-desk

# Setup client
npm create vite@latest client -- --template react
cd client
npm install
npm install axios react-router-dom firebase react-dropzone
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
cd ..

# Setup server
mkdir server && cd server
npm init -y
npm install express mongoose cors dotenv multer
npm install firebase-admin cloudinary @anthropic-ai/sdk
npm install nodemailer pdfjs-dist
npm install -D nodemon
cd ..
```

Running Locally

Start both servers:

```
# Terminal 1 — Start backend
cd server && npm run dev
# Server runs on http://localhost:5000

# Terminal 2 — Start frontend
cd client && npm run dev
# App runs on http://localhost:5173
```

10.3 Deployment

Deploy Backend to Render.com

61. Go to render.com → sign up free with GitHub
62. New → Web Service → Connect your GitHub repo
63. Root Directory: server
64. Build Command: npm install
65. Start Command: node app.js
66. Add all server environment variables in the Environment section
67. Deploy — Render gives you a URL like: https://iep-desk-api.onrender.com
68. Save this URL as VITE_API_URL in client .env

Deploy Frontend to Vercel

69. Go to vercel.com → sign up free with GitHub
70. New Project → Import your GitHub repo
71. Root Directory: client
72. Framework Preset: Vite
73. Add all VITE_* environment variables in the Environment Variables section
74. Deploy — Vercel gives you a URL like: https://iepdesk.vercel.app
75. Save this URL as CLIENT_URL in server Render environment

11. MVP Build Order

Build features in this exact order. Do not move to the next step until the current one is complete and tested. This order minimizes dependencies and ensures each step builds on a working foundation.

11.1 Build Sequence

#	Feature	What to Build	Test By
1	Project Setup	Create folder structure, install packages, connect MongoDB, set up env vars	Server starts, DB connects
2	Firebase Auth	Set up Firebase on client, create AuthContext, protect routes, verify token on server	Login with Google works
3	User Model & Auth Route	User Mongoose model, POST /api/auth/me creates user on first login	User saved to MongoDB
4	Child Profile	Profile model, full CRUD routes, Profile page with form, DiagnosisSelector component	Create and view profile
5	OpenRouter Service	openrouter.service.js with fallback logic, test with simple prompt	AI returns valid JSON
6	Rights Engine	Rights prompt, /api/rights route, Rights page UI with all result sections	Rights display for ADHD Texas
7	Cloudinary Setup	Cloudinary config, upload_stream helper, test file upload	PDF uploads to Cloudinary
8	IEP Analyzer	PDF.js text extraction, /api/analyze route, Analysis model, Analyzer page with upload zone and results	Upload IEP and see analysis
9	Meeting Prep	Meeting prep prompt, /api/meeting-prep route, MeetingPrep page with all sections	Generate kit from analysis
10	Document Vault	Document model, CRUD routes, Documents page with upload, list, delete	Upload and delete document
11	Dashboard	Dashboard page with profile card, quick actions, recent analyses	All links work correctly
12	Admin Panel	Admin middleware, admin routes, AdminDashboard, AdminUsers pages	Stats visible, can change roles
13	Landing Page	Full public landing page with all sections, waitlist email capture	Looks professional on mobile
14	Settings	Settings page — update name, photo, delete account	Photo uploads to Cloudinary

15	Deploy	Deploy server to Render, client to Vercel, test full flow on production	Everything works on live URL

11.2 Post-MVP Features (Build After First 100 Users)

Do not build these during MVP. Only add them after real users validate demand.

- Progress Tracker — log and visualize progress toward IEP goals over time
- Deadline Tracker — auto-detect and remind about annual review dates
- Accommodations Directory — browse 400+ accommodations filtered by diagnosis
- IEP Glossary — searchable database of all IEP terms in plain English
- Parent Community — forum for IEP Desk users to share experiences
- School District Database — state and district specific rules and precedents
- Mobile App — iOS and Android version of the web app
- Email Reminders — automated weekly progress nudges via Resend

12. Security Requirements

IEP Desk handles sensitive information about children with disabilities. Security is non-negotiable. The following requirements must be implemented before any user data is accepted.

12.1 Authentication Security

- All API routes except health check must require valid Firebase token
- Firebase Admin SDK must verify token on every request — never trust client-sent user IDs
- Admin routes must check user.role === 'admin' after auth check
- Never store Firebase tokens in localStorage — use memory or httpOnly cookies

12.2 Data Security

- All MongoDB queries must filter by userId — users can only access their own data
- Before returning any document, profile, or analysis — verify it belongs to the requesting user
- Cloudinary folders organized by userId — /iep-desk/{userId}/ — prevents access confusion
- Never log or store raw IEP text beyond what is needed for the analysis cache
- Add FERPA disclaimer to all pages that handle student educational records

12.3 Input Validation

- Validate all incoming request bodies on the server — never trust client data
- File upload: only allow PDF, PNG, JPG, JPEG — reject all other file types
- File size limit: maximum 10MB per upload
- Text inputs: sanitize all strings before saving to MongoDB
- Rate limit all AI endpoints: maximum 10 requests per user per hour

12.4 Legal Disclaimers (Required on Every AI Output)

This exact disclaimer must appear on every AI-generated response:

"This analysis is for informational and educational purposes only.
It does not constitute legal advice and does not create an attorney-client relationship.
Always verify information against your original IEP document.
For complex legal disputes, consult a qualified special education attorney or advocate."

13. Complete Package List

13.1 Server Packages

Package	Purpose
express	Web framework — routing, middleware
mongoose	MongoDB object modeling
cors	Cross-origin resource sharing headers
dotenv	Load environment variables from .env file
multer	Handle file uploads in memory
firebase-admin	Verify Firebase auth tokens on server
cloudinary	Upload and manage files in Cloudinary
nodemailer OR resend	Send transactional emails
express-rate-limit	Rate limit API routes to prevent abuse
helmet	Set security HTTP headers
nodemon (dev)	Auto-restart server on file changes during development

13.2 Client Packages

Package	Purpose
react	Core React library
react-router-dom	Client-side routing — all page navigation
firebase	Firebase Auth on the frontend
axios	HTTP requests to the Express API
react-dropzone	Drag and drop file upload component
pdfjs-dist	Extract text from PDF files client-side
recharts	Charts in admin dashboard
lucide-react	Icon library — clean minimal icons
tailwindcss (dev)	Utility-first CSS framework for all styling

14. Design System

All UI must follow these design specifications exactly. Keep it minimal, clean, and professional. No heavy gradients, no complex animations, no dark mode for MVP.

14.1 Colors

Name	Hex Value	Use
Primary	#2563EB	Buttons, links, active states, headings
Primary Dark	#1D4ED8	Button hover state
Primary Light	#EFF6FF	Info boxes, card backgrounds
Success	#16A34A	Success badges, positive scores
Warning	#D97706	Warning boxes, medium scores
Danger	#DC2626	Error states, red flags, low scores
Gray 50	#F9FAFB	Page background
Gray 100	#F3F4F6	Card alternating rows
Gray 800	#1F2937	Primary text color
White	#FFFFFF	Card backgrounds, button text

14.2 Typography

Element	Size	Style
Page Title (H1)	30px	Bold, Gray 900, font-weight 700
Section Title (H2)	24px	Bold, Blue #2563EB, font-weight 700
Card Title (H3)	18px	SemiBold, Gray 800, font-weight 600
Body Text	16px	Regular, Gray 700, font-weight 400
Small Text	14px	Regular, Gray 500, used for labels and metadata
Tiny Text	12px	Regular, Gray 400, used for disclaimers

14.3 Component Rules

- Buttons: rounded-lg (8px border radius), padding px-4 py-2.5, always show loading spinner when submitting
- Cards: white background, rounded-xl (12px), shadow-sm, border border-gray-200
- Inputs: border border-gray-300, rounded-lg, px-4 py-3, focus ring blue-500
- Tables: alternating row colors (white/gray-50), border-collapse, rounded corners on outer container
- Badges: small rounded-full, px-2.5 py-0.5, color coded by severity
- Loading states: always show spinner — never leave user with blank screen
- Empty states: always show message and CTA when list is empty
- Error states: red bordered box with clear error message and retry option

15. Quick Reference Cheat Sheet

Cut this page out and keep it beside you while coding.

Free Service URLs

Service	URL
MongoDB Atlas	mongodb.com/atlas
Firebase Console	console.firebase.google.com
OpenRouter	openrouter.ai
Cloudinary	cloudinary.com
Resend Email	resend.com
Vercel (Frontend)	vercel.com
Render (Backend)	render.com

Key Rules — Never Break These

Critical Rules for Every Developer Working on This Project

1. NEVER expose API keys in client-side code — all AI and DB calls go through the Express server
2. NEVER trust userId from the client — always use req.user._id set by auth middleware
3. NEVER skip the legal disclaimer — every AI response must include it before returning to frontend
4. NEVER store raw IEP text in plain text logs — treat all IEP content as sensitive PII
5. ALWAYS filter MongoDB queries by userId — data isolation is non-negotiable
6. ALWAYS validate file type before uploading — only accept PDF, PNG, JPG, JPEG
7. ALWAYS implement fallback when OpenRouter primary model fails
8. ALWAYS return proper error messages with HTTP status codes — never just 500

AI Model Priority Order

76. meta-llama/llama-4-maverick:free — Primary model for all tasks
77. google/gemini-2.0-flash-exp:free — Backup if primary fails
78. mistral/mistral-7b-instruct:free — Emergency fallback

Build Order Summary

79. Setup + MongoDB connect
80. Firebase Auth frontend + backend middleware
81. User model + auth route
82. Child profile CRUD
83. OpenRouter AI service
84. Rights Engine feature
85. Cloudinary file upload
86. IEP Analyzer feature
87. Meeting Prep feature
88. Document Vault
89. Dashboard page
90. Admin Panel
91. Landing Page
92. Settings Page
93. Deploy to Vercel + Render